

Gusto Meets — Google AI Studio Master Prompt Pack

Version: 1.0 · June 2026
Platform: Android (Kotlin) · Architecture: MVVM + Clean Architecture
AI Tool: Google AI Studio (office sessions) → Claude Code CLI (personal laptop, final push)

How to use this document

Paste one **Session Prompt** at a time into Google AI Studio. Each session is self-contained. After each session, save all generated files. When you are on your personal laptop, integrate and push to GitHub via Claude Code CLI.

Do not paste all sessions at once. AI Studio has a context limit and multi-session structure gives you checkpoints to review output quality.

Architecture Overview (for your reference before prompting)

Design Patterns (drawn from LLD principles)

Pattern	Applied To	Kotlin Implementation
Strategy	PricingStrategy, BookingStrategy	<code>interface</code> with <code>HourlyPricingStrategy</code> , <code>PurposeMultiplierStrategy</code> impls
Repository	TerraceRepo, BookingRepo, UserRepo, WalletRepo	Repository classes with Room (local) + Supabase (remote)
Singleton	BookingService, WalletService	Hilt <code>@Singleton</code> scope (replaces manual singleton)
Factory	TerraceFactory (space type creation)	Sealed class + companion factory functions

Pattern	Applied To	Kotlin Implementation
Observer	Host dashboard live updates	Supabase Realtime / StateFlow + collectAsState

Concurrency (adapted from AtomicBoolean pattern in car rental LLD)

The car rental video used Java's `AtomicBoolean.compareAndSet()` to prevent double-booking. In Kotlin + Coroutines:

- `Mutex` in the BookingService replaces AtomicBoolean — only one coroutine holds the booking lock at a time
- Supabase row-level `SELECT FOR UPDATE` (pessimistic locking) on the bookings table prevents race conditions from different devices
- Optimistic locking via a `version` column on `terraces` table (incremented on every booking) catches simultaneous updates

Payment Timer (adapted from hotel booking Redis 15-minute timer)

The hotel booking video used Redis cluster for the 15-minute payment timeout. In Android:

- `WorkManager` + a `PENDING_PAYMENT` booking status handles the timer (survives app background/kill)
- If payment not completed in 15 min → WorkManager job sets `status = CANCELLED` and restores inventory

Data Layer

Remote: Supabase (PostgreSQL) — source of truth

Local: Room DB — offline cache + active booking state

Sync: Supabase Realtime WebSocket — host dashboard live updates

Cache: DataStore Preferences — wallet balance, session token

Session 1 — Project Setup + Architecture Scaffold

CONTEXT:

I have an existing Android Kotlin project with a standard Gradle structure

(app/src/main/java, app/src/main/res, build.gradle.kts). I am building

"Gusto Meets" — an Airbnb-style terrace rental marketplace app where users can book residential terraces for private socializing by the hour.

TASK:

Update the existing Android project to industry-standard MVVM + Clean Architecture. Do not delete existing files — add to them.

ARCHITECTURE LAYERS:

1. presentation/ — Jetpack Compose UI screens + ViewModels
2. domain/ — Use cases, entities, repository interfaces
3. data/ — Repository implementations, Room DB, Supabase remote sources
4. di/ — Hilt dependency injection modules

REQUIRED DEPENDENCIES (add to app/build.gradle.kts):

- Jetpack Compose BOM (latest stable)
- Hilt (com.google.dagger:hilt-android)
- Room (androidx.room:room-ktx)
- Retrofit + OkHttp (for Supabase REST calls)
- Kotlin Coroutines + Flow
- Coil (image loading)
- Jetpack Navigation Compose
- WorkManager (androidx.work:work-runtime-ktx)
- DataStore Preferences
- Razorpay Android SDK (com.razorpay:checkout:1.6.40)

- Firebase BoM + Firebase Auth + Firebase Messaging

- Google Play Services Auth (for Google OAuth)

PLACEHOLDER API KEYS (create a Constants.kt file with these — all as TODO strings):

```
```kotlin
```

```
object AppConstants {
```

```
 // Supabase
```

```
 const val SUPABASE_URL = "TODO_REPLACE_WITH_SUPABASE_URL"
```

```
 const val SUPABASE_ANON_KEY = "TODO_REPLACE_WITH_SUPABASE_ANON_KEY"
```

```
 // Firebase (also needs google-services.json — add placeholder comment)
```

```
 // TODO: Replace google-services.json in app/ directory with real file from Firebase Console
```

```
 // Razorpay
```

```
 const val RAZORPAY_KEY_ID = "TODO_REPLACE_WITH_RAZORPAY_KEY_ID"
```

```
 // Note: Razorpay Secret Key must NEVER be in the Android app — server-side only
```

```
 // Aadhaar KYC (Setu API)
```

```
 const val SETU_KYC_BASE_URL = "https://dg-sandbox.setu.co" // sandbox
```

```
 const val SETU_CLIENT_ID = "TODO_REPLACE_WITH_SETU_CLIENT_ID"
```

```
 const val SETU_CLIENT_SECRET = "TODO_REPLACE_WITH_SETU_CLIENT_SECRET"
```

```

// WhatsApp Business API (via Interakt)

const val INTERAKT_API_KEY = "TODO_REPLACE_WITH_INTERAKT_API_KEY"

// Google OAuth Web Client ID (from Firebase Console > Authentication > Google)

const val GOOGLE_OAUTH_WEB_CLIENT_ID =
"TODO_REPLACE_WITH_WEB_CLIENT_ID"

// Platform settings

const val PLATFORM_FEE_PERCENT = 0.15 // 15%

const val PAYMENT_TIMEOUT_MINUTES = 15L

const val OVERSTAY_MULTIPLIER = 2.0 // 2x hourly rate for overstay penalty

const val BOOKING_BUFFER_MINUTES = 30 // gap between bookings

}

```

CREATE these domain entity data classes:

- User(id, phoneNumber, fullName, role[GUEST/HOST/ADMIN], kycVerified, walletBalance)
- Terrace(id, hostId, title, description, addressLine, area, city, geoLat, geoLng, accessCategory, maxCapacity, baseHourlyRate, minBookingHours, photos[List], verificationStatus, isActive, permissions: TerracePermissions)
- TerracePermissions(allowAlcohol, allowSmoking, allowLoudMusic, allowOutsideFood, allowCouples, allowedPurposes[List], partyMultiplier, alcoholDeposit)
- Booking(id, guestId, terraceId, startTime, endTime, actualCheckoutTime, purpose, guestCount, status[BookingStatus enum], hourlyRateApplied, totalTimeCost, securityDepositHeld, platformFee, overstayPenalty, razorpayOrderId, digitalWaiverSigned, hostCheckedIn)
- WalletTransaction(id, userId, amount, balanceAfter, description, bookingId, createdAt)

CREATE these enums:

- UserRole: GUEST, HOST, ADMIN

- BookingPurpose: CHILLOUT, DINE\_OUT, MOVIE\_NIGHT, PARTY, BOARD\_GAMES, STUDY\_GROUP
- BookingStatus: PENDING\_PAYMENT, CONFIRMED, ACTIVE, EXTENDED, COMPLETED, OVERSTAYED, DISPUTED, CANCELLED
- AccessType: PRIVATE\_STAIRCASE, SHARED\_WALKTHROUGH
- VerificationStatus: UNVERIFIED, PENDING\_INSPECTION, VERIFIED, SUSPENDED

CREATE repository interfaces in domain/repository/:

- UserRepository
- TerraceRepository
- BookingRepository
- WalletRepository

OUTPUT: Updated build.gradle.kts, Constants.kt, all entity files, all enum files, all repository interface files, basic Hilt AppModule stub. Show full file content for each.

---

## ## Session 2 — Authentication (OTP + Google OAuth)

CONTEXT: Continuing Gusto Meets Android app. Architecture is MVVM + Clean Architecture with Hilt DI. Firebase Auth is already in dependencies. Users authenticate via (1) Phone number OTP and (2) Google Sign-In.

TASK: Build the complete authentication flow.

SCREENS TO CREATE (Jetpack Compose):

1. SplashScreen — check if user is logged in → route to Home or Auth
2. AuthScreen — phone number input (Indian format +91, with validation) + "Send OTP" button + Google Sign-In button
3. OtpVerificationScreen — 6-digit OTP input with auto-advance between fields, 60-second resend timer, verify button

FIREBASE OTP FLOW:

- Use PhoneAuthProvider.verifyPhoneNumber() with +91 prefix
- Handle three callbacks: onVerificationCompleted (auto-verify), onVerificationFailed (show error), onCodeSent (navigate to OTP screen)
- On OTP submit: signInWithCredential(PhoneAuthProvider.getCredential(verificationId, otp))
- On success: check if user document exists in Supabase users table

- If new user: create user record, navigate to ProfileSetupScreen
- If existing user: navigate to HomeScreen

#### GOOGLE OAUTH FLOW:

- Use credential manager API (androidx.credentials) — modern approach, not deprecated  
GoogleSignInClient
- GoogleIdTokenRequestOptions with GOOGLE\_OAUTH\_WEB\_CLIENT\_ID from Constants
- On token received: Firebase  
signInWithCredential(GoogleAuthProvider.getCredential(idToken, null))
- Same post-login routing as OTP flow

#### PROFILE SETUP SCREEN (shown only once, for new users):

- Full name input
- User role selection (Guest / Host) — radio buttons
- "Continue" button → saves to Supabase users table → navigates to HomeScreen

#### VIEWMODEL: AuthViewModel with:

- sendOtp(phoneNumber: String)
- verifyOtp(code: String)
- signInWithGoogle(context: Context)
- State: AuthUiState sealed class (Idle, Loading, OtpSent, Success, Error(message))

OUTPUT: SplashScreen.kt, AuthScreen.kt, OtpVerificationScreen.kt, ProfileSetupScreen.kt, AuthViewModel.kt, FirebaseAuthDataSource.kt (data layer), UserRepositoryImpl.kt stub showing how Firebase result maps to domain User entity. Show full file content for each.

---

#### ## Session 3 — Aadhaar KYC Flow

CONTEXT: Continuing Gusto Meets. Auth is complete. We now need Aadhaar KYC verification for guests before their first booking. This uses the Setu API (No-OTP Aadhaar demographic verification). All API keys are placeholders from Constants.kt.

#### BACKGROUND — How Setu No-OTP Aadhaar works:

1. Guest enters Aadhaar number (12 digits)
2. App runs Verhoeff checksum validation client-side (prevents bad API calls)

3. App calls our backend Edge Function (NOT the Setu API directly — API secret must never be in the Android app)
4. Backend Edge Function calls Setu's /aadhaar/verify endpoint with the Aadhaar number
5. Setu returns demographic data: name, year of birth, gender, state
6. Backend performs fuzzy name match: compare returned name against user's profile name
7. Backend returns: {verified: boolean, verifiedName: string, referenceToken: string}
8. App updates user's kyc\_verified = true, stores kyc\_reference\_token (NOT the Aadhaar number)

TASK: Build the KYC screen and supporting code.

VERHOEFF ALGORITHM (implement this in Kotlin): The Verhoeff algorithm validates that a 12-digit number is mathematically valid before making an API call. Implement as VerhoeffValidator.kt with a static validate(number: String): Boolean function using the standard 3 tables (multiplication, permutation, inverse).

KYC SCREEN:

- Title: "Verify your identity"
- Subtitle: "Required once before your first booking. Your Aadhaar number is never stored — only a verification token."
- Aadhaar input: 12-digit numeric, formatted as XXXX-XXXX-XXXX for display, real-time Verhoeff check (show green tick or red X below input)
- Full name input: pre-filled from user profile, editable
- Consent checkbox: "I consent to demographic verification via UIDAI-licensed gateway"
- Verify button: disabled until Verhoeff passes + name filled + consent checked

API CALL (placeholder — real backend needed): Create KycApiService.kt using Retrofit with base URL from Constants.SETU\_KYC\_BASE\_URL. For now, create a MOCK implementation that returns success after 2 seconds — add a clear comment: // TODO: Replace MockKycDataSource with real Supabase Edge Function call

VIEWMODEL: KycViewModel with:

- State: KycUiState (Idle, ValidatingLocally, Submitting, Success(verifiedName), Error(msg))
- fun onAadhaarChanged(input: String) — runs Verhoeff, updates state
- fun submitKyc(aadhaarNumber: String, fullName: String)

ON SUCCESS:

- Update user record in Supabase: kyc\_verified = true, kyc\_reference\_token = token



- Store kyc\_verified = true in DataStore (so future booking flows don't re-check remotely)
- Navigate to HomeScreen

OUTPUT: KycScreen.kt, KycViewModel.kt, VerhoeffValidator.kt, KycApiService.kt (Retrofit interface), MockKycDataSource.kt (temporary mock), updated UserRepositoryImpl.kt showing kyc update. Show full file content for each.

---

## ## Session 4 — Core Booking Flow with Strategy Pattern

CONTEXT: Continuing Gusto Meets. Auth and KYC are complete. Now building the core terrace discovery and booking flow. We apply the Strategy Pattern and Repository Pattern from LLD principles — inspired by how Car Rental systems use strategies for pricing and booking selection.

### DESIGN PATTERNS TO IMPLEMENT:

1. PricingStrategy (Strategy Pattern): interface PricingStrategy { fun calculateTotal(baseRate: Double, hours: Double, purpose: BookingPurpose, permissions: TerracePermissions): PricingBreakdown }

```
data class PricingBreakdown(val baseAmount: Double, val purposeMultiplier: Double, val
purposePremium: Double, val platformFee: Double, val securityDeposit: Double, val
totalAmount: Double)
```

### Implementations:

- StandardPricingStrategy: base \* hours \* purposeMultiplier + 15% platform fee
  - PeakHourPricingStrategy: StandardPricing + 20% if Friday/Saturday 6pm-11pm (Phase 2 — stub only)
2. BookingStrategy (Strategy Pattern): interface BookingConflictStrategy { fun findAvailableSlot( terracelId: String, requestedStart: ZonedDateTime, requestedEnd: ZonedDateTime, existingBookings: List): SlotAvailabilityResult } sealed class SlotAvailabilityResult { object Available : SlotAvailabilityResult() data class Unavailable(val nextAvailableSlot: ZonedDateTime?) : SlotAvailabilityResult() }

Implementation: BufferedSlotStrategy — checks if requested slot + 30-min buffer overlaps any CONFIRMED/ACTIVE/EXTENDED/OVERSTAYED bookings.

3. Concurrency (from Car Rental AtomicBoolean pattern, adapted for Kotlin): In BookingService, use a Mutex to prevent two simultaneous booking attempts for the

```
same terrace slot: private val bookingMutex = Mutex() suspend fun bookTerrace(...):
BookingResult = bookingMutex.withLock { ... }
```

## SCREENS TO BUILD:

### HomeScreen:

- Search bar (city + date)
- "Terraces near you" horizontal scroll cards
- Filter chips: All / Chillout / Dine Out / Party / Board Games
- Each TerraceCard: photo, title, area, ₹X/hr, capacity, permission icons

### TerraceDetailScreen (route: /terrace/{terraceId}):

- Photo pager (swipeable)
- Host name + rating
- Permission matrix (icon grid: food, alcohol, smoking, couples, music toggles)
- "About this space" expandable text
- Booking section:
  - Date picker (calendar)
  - Time slot picker (from/to, 30-min increments, blocked slots shown in gray)
  - Purpose selector (chips: Chillout, Dine Out, Party, etc.)
  - Guest count stepper
  - LIVE pricing breakdown card — updates in real-time as user changes selections:  
"Base: ₹200 × 2 hrs = ₹400 | Party premium (1.3×): +₹120 | Platform fee: +₹78 |  
Deposit: ₹500 | Total: ₹1,098"
  - "Proceed to Book" button (disabled if slot unavailable, greyed with "Slot taken")

### BookingSummaryScreen:

- Shows full breakdown
- Digital waiver text (scrollable) + checkbox "I accept"
- KYC verification badge (shows name if verified, "Verify ID" link if not)
- "Pay ₹X,XXX" button — triggers Razorpay

### VIEWMODEL: TerraceDetailViewModel with:

- loadTerrace(terraceId: String)
- updateBookingParams(date, startTime, endTime, purpose, guestCount)
- checkSlotAvailability() — uses BufferedSlotStrategy
- calculatePrice() — uses StandardPricingStrategy
- State: TerraceDetailUiState

OUTPUT: HomeScreen.kt, TerraceDetailScreen.kt, BookingSummaryScreen.kt, TerraceDetailViewModel.kt, PricingStrategy.kt (interface + StandardPricingStrategy), BookingConflictStrategy.kt (interface + BufferedSlotStrategy), BookingService.kt (with Mutex concurrency), TerraceRepositoryImpl.kt stub. Show full file content for each.

---

## ## Session 5 — Razorpay Integration + Active Booking

CONTEXT: Continuing Gusto Meets. Core booking flow is built. Now integrating Razorpay payment and the active booking experience (timer, extend, checkout).

### RAZORPAY ANDROID SDK INTEGRATION:

1. PaymentService.kt — wraps Razorpay Checkout:
  - initiate(activity: Activity, bookingId: String, amount: Int, userPhone: String, userName: String)
  - Razorpay options:
    - key: Constants.RAZORPAY\_KEY\_ID
    - amount: in paise (multiply rupees × 100)
    - name: "Gusto Meets"
    - description: "Terrace booking - {bookingId}"
    - prefill: phone, name
    - theme.color: "#10B981" (Gusto teal)
    - modal.backdropclose: false (prevent accidental close)
  - Implements PaymentResultWithDataListener
  - onPaymentSuccess → emit RazorpayResult.Success(paymentId, orderId)
  - onPaymentError → emit RazorpayResult.Failure(code, description)
2. POST-PAYMENT FLOW (adapted from hotel booking 15-min timer + rollback logic): On Razorpay success:
  - Call BookingRepository.confirmPayment(bookingId, razorpayPaymentId)
  - Backend (Supabase Edge Function stub) updates booking status = CONFIRMED
  - Deduct total from wallet\_transactions ledger
  - Navigate to ActiveBookingScreen

On Razorpay failure:

- BookingRepository.cancelBooking(bookingId)
- Update booking status = CANCELLED (do NOT deduct wallet — no payment taken)
- Show error with retry option
- Note in comment: // Pattern: same as car rental rollback — isBooked must reset to false on payment failure

3. PAYMENT TIMEOUT (WorkManager — replaces Redis timer from hotel booking design): When booking status = PENDING\_PAYMENT, schedule a WorkManager OneTimeWorkRequest with a 15-minute delay. Name: "payment\_timeout\_{bookingId}". If it fires before payment: cancel booking, update status = CANCELLED. If payment succeeds: cancel the WorkManager job by name.

ACTIVE BOOKING SCREEN (route: /booking/active/{bookingId}):

- Live countdown timer: "2:43:17 remaining" — large, prominent, ticking Implementation: use LaunchedEffect + delay(1000) loop, compute from booking.endTime - now()
- Terrace name + area + full address
- "Navigate" button: opens Google Maps intent with terrace coordinates
- Host name + phone number (click to call intent)
- "Extend Session" FAB button → shows bottom sheet:
  - Options: +30 min (₹XX), +1 hr (₹XX), +2 hrs (₹XX)
  - Calculates cost using same PricingStrategy
  - "Pay & Extend" → Razorpay for extension amount → BookingRepository.extendBooking()
  - On success: update booking.endTime, restart WorkManager timeout job
- "Check Out Early" text button → confirmation dialog → BookingRepository.checkoutBooking()
  - Sets actual\_checkout\_time = now()
  - Refunds unused time proportionally from wallet (if any)
  - Navigate to BookingCompletedScreen

T-MINUS 15 ALERT: Use LaunchedEffect in ActiveBookingScreen to detect when remaining time < 15 minutes. Show a non-dismissible bottom sheet: "Your session ends in 15 minutes. Extend or prepare to leave."

BookingCompletedScreen:

- "Thanks for visiting!"
- Rating prompt: 1-5 star tap for host (submits to reviews table)
- "Book Again" button

OUTPUT: PaymentService.kt, ActiveBookingScreen.kt, BookingCompletedScreen.kt, ExtendSessionBottomSheet.kt, PaymentTimeoutWorker.kt (WorkManager), BookingRepositoryImpl.kt (complete with confirmPayment, cancelBooking, extendBooking, checkoutBooking). Show full file content for each.

---

## Session 6 — Host Dashboard + Wallet + Admin

CONTEXT: Continuing Gusto Meets. Guest flow is complete. Now building host-side experience, wallet management, and admin tools.

HOST DASHBOARD SCREEN (role-gated: visible only to HOST role users):

1. TodayBookingsTab:
  - List of today's confirmed bookings as cards
  - BookingCard shows: guest verified badge (KYC), guest name, group size, purpose chip, time slot, pricing summary
  - "Check In Guest" button:
    - Sends deep link / QR code to guest via WhatsApp (stub — print log for now)
    - Updates booking.host\_checked\_in = true, booking.status = ACTIVE
    - Button changes to "Guest Checked In" with green badge
  - Supabase Realtime subscription: Subscribe to bookings table WHERE terrace\_id IN host's terraces
    - Use Supabase Kotlin client's .from("bookings").on(INSERT) to show toast "New booking received!" when a new confirmed booking arrives
    - Pattern: same as hotel booking's WebSocket real-time updates
2. ActiveNowTab:
  - Currently active bookings with live timer (same countdown component as guest)
  - "Report Damage" button → opens DamageReportBottomSheet:
    - Description text field
    - Photo picker (up to 5 photos — uses Android photo picker API)
    - "Estimated damage amount" number input
    - Submit → creates damage\_report record, freezes guest's security deposit
3. HistoryTab:
  - Past completed bookings, paginated (20 per page)
  - Shows total earnings summary: "₹X,XXX this month"

WALLET SCREEN (accessible to both GUEST and HOST):

- Current balance: large, prominent "₹ 2,450.00"
- "Add Money" button → Razorpay checkout for top-up (amounts: ₹200, ₹500, ₹1000, ₹2000, or custom) On success: update users.wallet\_balance, create wallet\_transaction credit entry
- Transaction history list from wallet\_transactions table:
  - Each row: date, description, amount (green for credit, red for debit), balance\_after
  - Paginated (load more button)

ADMIN SCREEN (role-gated: ADMIN only — add a hidden navigation path): Three tabs matching the three admin responsibilities:

#### TerraceVerificationTab:

- List terraces WHERE verification\_status = 'PENDING\_INSPECTION'
- Each card: terrace title, area, host name, parapet height, safety video URL (opens browser)
- "Approve" button → update verification\_status = VERIFIED, is\_active = true
- "Reject" button → update verification\_status = SUSPENDED, prompts for rejection note

#### DisputesTab:

- List damage\_reports WHERE status = 'PENDING'
- Shows: booking context, guest photo, host photo evidence, claimed amount
- "Approve Claim" → capture security deposit from Razorpay pre-auth, route to host wallet (Mark as resolved, update damage\_report.resolved\_amount)
- "Reject Claim" → refund deposit to guest wallet

#### OverstaysTab:

- List bookings WHERE status = 'OVERSTAYED'
- Manual override: Admin can "Force Checkout" which triggers the same penalty calculation as the automated overstay function

OVERSTAY AUTOMATED HANDLER (WorkManager): Create OverstayCheckWorker.kt — a periodic WorkManager job running every 5 minutes.

- Query Room DB for bookings WHERE status = ACTIVE AND endTime < now()
- For each: update status = OVERSTAYED
- Calculate penalty: (minutesOverstayed / 60.0 ceiling) × hourlyRate × OVERSTAY\_MULTIPLIER
- Create WalletTransaction debit on guest
- Create WalletTransaction credit on host (their portion)
- Send WhatsApp notification to guest (stub: log message)
- Pattern from car rental LLD: same as payment failure rollback — system enforces state consistency without human intervention

NAVIGATION GRAPH (update NavGraph.kt to include all routes):

- splashScreen → authScreen → otpScreen → profileSetupScreen → homeScreen
- homeScreen → terraceDetailScreen → bookingSummaryScreen → activeBookingScreen → bookingCompletedScreen
- homeScreen → walletScreen
- homeScreen → kycScreen (triggered when booking attempted without KYC)

- homeScreen → hostDashboardScreen (HOST role only)
- homeScreen → adminScreen (ADMIN role — hidden, accessed via settings)

OUTPUT: HostDashboardScreen.kt, WalletScreen.kt, AdminScreen.kt (all 3 tabs), DamageReportBottomSheet.kt, OverstayCheckWorker.kt, WalletRepositoryImpl.kt, NavGraph.kt (complete navigation graph connecting all sessions). Show full file content for each.

---

## ## Session 7 — Final Polish: Error Handling, Offline, Performance

CONTEXT: Final session for Gusto Meets. All core flows are built. This session handles production-readiness: error handling, network states, loading skeletons, optimistic locking, and security hardening.

TASK 1 — Network State Handling: Create NetworkMonitor.kt using ConnectivityManager.NetworkCallback. Inject into all ViewModels via Hilt. Show a persistent "No internet — some features may not work" banner in Scaffold when offline. Offline-capable screens (using Room cache): Home/Explore, Booking History, Wallet History. Online-required screens (show error if offline): New booking, Payment, KYC.

TASK 2 — Optimistic Locking on Bookings (from hotel booking system design): The hotel booking video demonstrated optimistic locking via a version column. In Gusto Meets, implement this at the Supabase level: Add a version INT column to the bookings table (migration note as a comment). When BookingRepositoryImpl.confirmBooking() is called, include: WHERE id = bookingId AND version = expectedVersion If 0 rows affected → another device updated the booking → throw ConcurrentModificationException Show user: "Booking was modified elsewhere. Please refresh and try again."

TASK 3 — Loading States + Skeleton UI: For HomeScreen terrace cards: show 4 shimmer skeleton cards while loading. Create ShimmerBox composable using Jetpack Compose animateFloat with Brush.linearGradient alternating between gray-100 and gray-300.

TASK 4 — Security Hardening: Create SecurityConfig.kt:

- Network Security Config: add res/xml/network\_security\_config.xml with certificate pinning for Supabase domain (placeholder — comment shows where to add pins)
- ProGuard rules: add rules to keep Razorpay, Firebase, Hilt, Room generated classes
- Update AndroidManifest.xml:
  - android:networkSecurityConfig reference
  - Remove android:debuggable from release build config
  - Add INTERNET, ACCESS\_NETWORK\_STATE permissions
  - Add queries block for WhatsApp, Google Maps, Phone call intents

TASK 5 — Analytics Events (Firebase Analytics — placeholder stubs): Create AnalyticsService.kt with these stub events (implement with Firebase Analytics):

- logEvent("terrace\_viewed", terraceId, area)
- logEvent("booking\_initiated", purpose, estimatedAmount)
- logEvent("booking\_confirmed", bookingId, amount)
- logEvent("payment\_failed", reason)
- logEvent("kyc\_completed")
- logEvent("session\_extended", additionalMinutes, additionalCost)

TASK 6 — README.md Update: Update the project README.md with:

- App overview (2 paragraphs)
- Setup instructions (Clone → Add google-services.json → Replace TODO constants → Run)
- Architecture diagram (ASCII art MVVM layers)
- API keys checklist (table of all TODO placeholders and where to get each)
- Known limitations / Phase 2 items (Elasticsearch search, peak pricing, host self-onboarding)

OUTPUT: NetworkMonitor.kt, ShimmerBox.kt (reusable composable), SecurityConfig.kt + network\_security\_config.xml + proguard-rules.pro updates, AnalyticsService.kt, updated README.md. Show full file content for each.

---

## Integration Checklist (for Claude Code CLI on personal laptop)

After running all 7 AI Studio sessions and collecting the generated files:

```
```bash
```

1. On personal laptop — create the repo

```
cd C:\Users\Adithya\Desktop
```

```
git init gusto-meets-android
```

```
cd gusto-meets-android
```

2. Copy all AI Studio generated files into the directory structure

3. Replace all TODO placeholders — do this BEFORE first run:

- Add real google-services.json to app/

- Replace SUPABASE_URL and SUPABASE_ANON_KEY in Constants.kt

- Replace RAZORPAY_KEY_ID (use test key: rzp_test_XXXXXXXXXX)

- Replace SETU_CLIENT_ID and SETU_CLIENT_SECRET (register at docs.setu.co)

- Replace GOOGLE_OAUTH_WEB_CLIENT_ID from Firebase Console

4. Run the Supabase migration SQL from gusto_meets_build_guide.md

(in Supabase dashboard > SQL Editor)

5. First build check

```
./gradlew assembleDebug 2>&1 | head -50
```

6. If build passes, commit

```
git add .
```

```
git commit -m "feat: complete Gusto Meets MVP — auth, KYC, booking, payments, host dashboard"
```

```
git remote add origin https://github.com/YOUR_USERNAME/gusto-meets-android.git
```

```
git push -u origin main
```

API Keys — Where to Get Each

Constant	Service	Registration URL	Free Tier
SUPABASE_URL + ANON_KEY	Supabase	supabase.com	✅ 500MB, 50k users
RAZORPAY_KEY_ID	Razorpay	dashboard.razorpay.com	✅ Test mode unlimited

Constant	Service	Registration URL	Free Tier
SETU_CLIENT_ID/SECRET	Setu Aadhaar API	docs.setu.co	✅ Sandbox free
GOOGLE_OAUTH_WEB_CLIENT_ID	Firebase Console	console.firebase.google.com	✅ Spark plan
INTERAKT_API_KEY	Interakt WhatsApp	interakt.shop	💰 Paid (skip for MVP)

MVP can launch without INTERAKT — replace WhatsApp notifications with Firebase Cloud Messaging push notifications (already in dependencies). WhatsApp API is a Phase 2 enhancement.

Phase 2 Features (do not build now — document for later)

- Elasticsearch integration for fuzzy terrace search (currently: simple LIKE query on Supabase)
- PeakHourPricingStrategy implementation (Friday/Saturday 6–11pm premium)
- Host self-onboarding with video upload + admin review queue
- In-app group chat between host and guest before check-in
- Gusto App integration: order food from partner restaurants and have it delivered to the terrace
- Web PWA companion (Next.js) using same Supabase backend — for users who don't want to install